of areas and operations, as mentioned earlier. In addition, since it is Free Software, you can tailor the code to perfectly suit any need.

Why use GSL?

GSL is a very attractive package for the following reasons:

- GSL has a wide range of functions available, right from matrix decomposition, to random number generators of different distributions—and many more. In fact, GSL has almost all the functions you need in order to make your work faster and easier; take a look at the list at http://www.gnu.org/software/gsl/.
- The library is written in ANSI C (C89), is thread-safe, and should be portable to any system with a working ANSI C compiler.
- You can also call GSL functions directly from C++.
- GSL does not have any dependencies on other packages, making it easy to use even on machines that do not have GSL installed. It's possible to copy GSL function and data type declarations from the GSL .c and .h files into your own program, or to distribute the GSL source along with your program, adjusting your Makefile (or creating another Makefile) to compile GSL before your program. This is possible because GSL is distributed under the GNU GPL; naturally, you couldn't do this in a proprietary program.
- GSL uses an object-oriented design, so different algorithms can be changed or plugged in at run-time without recompiling the program.

So, are you convinced now that GSL could be of use to you? Before we try writing a program with it, let's take a quick look at how to install GSL.

Installing GSL

The GSL home page (URL was mentioned earlier) has download links. Although your distribution probably has GSL available in its package repositories, I prefer to download the source tarball, and compile and install GSL with the standard set of commands in a terminal window. I would recommend that you, too, do this. This is for these reasons:

- Having the source available is a big help for a scientific programmer who's comfortable with C, since you can look at the source code to find out the definitions of various GSL variables, if you need to make advanced use of them (more below, after we do our first GSL program).
- You can perhaps also learn some programming tips from the GSL code; or even fix, enhance or customise the GSL code for your own purposes.
- As mentioned earlier, you can copy portions of GSL code into your FOSS program, or distribute GSL with your program; having the source at hand makes that easy.

Once you've downloaded the GSL source tarball, extract

```
File  Edit  View  Terminal  Help
amol@amol-desktop:~$ cd gsl-1.14/
amol@amol-desktop:~/gsl-1.14$ ./con
config.guess   config.status  config.sub      configure      const/
amol@amol-desktop:~/gsl-1.14$ ./configure
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a thread-safe mkdir -p... /bin/mkdir -p
checking for gawk... no
checking for mawk... mawk
checking whether make sets $(MAKE)... yes
checking whether to enable maintainer-specific portions of Makefiles... no
checking for a sed that does not truncate output... /bin/sed
checking whether make sets $(MAKE)... (cached) yes
checking build system type...
```

Figure 1: Running ./configure

```
File  Edit  View  Terminal  Help
~ mv -f .deps/gsl-randist.Tpo .deps/gsl-randist.Po
/bin/bash ../libtool --tag=CC   --mode=link gcc  -g -O2   -o gsl-randist gsl-rand
ist.o libgsl.la cblas/libgslcblas.la -lm
libtool: link: gcc -g -O2 -o .libs/gsl-randist gsl-randist.o  ./.libs/libgsl.so
cblas/.libs/libgslcblas.so -lm
/bin/bash ../libtool --tag=CC   --mode=link gcc  -g -O2   -o gsl-histogram gsl-hi
stogram.o libgsl.la cblas/libgslcblas.la -lm
libtool: link: gcc -g -O2 -o .libs/gsl-histogram gsl-histogram.o  ./.libs/libgsl
.so cblas/.libs/libgslcblas.so -lm
creating gsl-config
creating gsl.pc
make  test gsl_histogram.sh
make[2]: Entering directory `/home/amol/gsl-1.14'
make[2]: Nothing to be done for `test_gsl_histogram.sh'.
make[2]: Leaving directory `/home/amol/gsl-1.14'
make  check-TESTS
make[2]: Entering directory `/home/amol/gsl-1.14'
PASS: test_gsl_histogram.sh
==========
1 test passed
==========
make[2]: Leaving directory `/home/amol/gsl-1.14'
make[1]: Leaving directory `/home/amol/gsl-1.14'
~ amol@amol-desktop:~/gsl-1.14$
```

Figure 2: 'Make check' output

```
File  Edit  View  Terminal  Help
~ amol@amol-desktop:~/gsl-1.14$ sudo make install
Making install in gsl
make[1]: Entering directory `/home/amol/gsl-1.14/gsl'
make[2]: Entering directory `/home/amol/gsl-1.14/gsl'
make[2]: Nothing to be done for `install-exec-am'.
make[2]: Nothing to be done for `install-data-am'.
make[2]: Leaving directory `/home/amol/gsl-1.14/gsl'
make[1]: Leaving directory `/home/amol/gsl-1.14/gsl'
Making install in utils
make[1]: Entering directory `/home/amol/gsl-1.14/utils'
/bin/bash ../libtool --tag=CC   --mode=compile gcc -DHAVE_CONFIG_H -I. -I..    -g -
O2 -MT placeholder.lo -MD -MP -MF .deps/placeholder.Tpo -c -o placeholder.lo placeho
lder.c
libtool: compile:  gcc -DHAVE_CONFIG_H -I. -I.. -g -O2 -MT placeholder.lo -MD -MP -M
F .deps/placeholder.Tpo -c placeholder.c  -fPIC -DPIC -o .libs/placeholder.o
libtool: compile:  gcc -DHAVE_CONFIG_H -I. -I.. -g -O2 -MT placeholder.lo -MD -MP -M
F .deps/placeholder.Tpo -c placeholder.c -o placeholder.o >/dev/null 2>&1
mv -f .deps/placeholder.Tpo .deps/placeholder.Plo
/bin/bash ../libtool --tag=CC   --mode=link gcc  -g -O2   -o libutils.la  placeholde
r.lo  -lm
libtool: link: ar cru .libs/libutils.a .libs/placeholder.o
libtool: link: ranlib .libs/libutils.a
libtool: link: ( cd ".libs" && rm -f "libutils.la" && ln -s "../libutils.la" "libuti
ls.la" )
```

Figure 3: Running 'make install'

the contents of the archive with the following command:

```
$ tar -xvzf gsl-1.14.tar.gz
```

Change your working directory into the archive-extracted directory:

```
$ cd gsl-1.14
```

It is recommended that you use the latest version of